Temas Selectos de Física Computacional III: Introducción a la Ciencia de Datos. Clase 9: 07 de octubre de 2026

Dr. Jaime Octavio Guerra Pulido

1 Introducción

En esta clase, se estudiarán los problemas de clasificación y cómo pueden abordarse utilizando una categoría de modelos basados en árboles de decisión de Scikit-learn usando el conjunto de datos del vino.

2 Arboles de decisión

Se usará el conjunto de datos del vino para clasificar los vinos en tres categorías (usando 13 características físicas del vino). A partir de estos 13 componentes (características), el objetivo es predecir la clase (target) del vino. Puedes descargar el conjunto de datos desde UCI o puedes obtenerlo directamente desde Scikit-learn:

```
import pandas as pd
from sklearn.datasets import load_wine
dataset = load_wine()
df = pd.DataFrame(data=dataset["data"], columns=dataset["feature_names"])
df["target"] = dataset["target"]
```

Primero, recordemos que para contar por clase y el balance de las clases, podemos hacer:

```
class_counts = df['target'].value_counts().sort_index()
print("\nPorcentaje por clase:")
print((class_counts / class_counts.sum()) * 100)
```

Después, haz nuevamente los gráficos con pairplot y heatmap de seaborn.

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="ticks")
sns.pairplot(df, hue='target', vars=df.columns[:-1])
```

```
#cosas bonitas: corner=True, plot_kws={'alpha':0.6},palette='Set1')
plt.show()

corr = df.drop(columns=['target']).corr()
plt.figure(figsize=(12,10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap='coolwarm', square=True)
plt.show()
```

Los **árboles de decisión son algoritmos de aprendizaje supervisado** utilizados tanto para tareas de clasificación como de regresión.

La idea principal de los árboles de decisión es encontrar aquellas características descriptivas que contienen la mayor cantidad de **información** con respecto a la característica objetivo, y luego dividir el conjunto de datos según los valores de estas características de tal manera que los valores de la característica objetivo para los subconjuntos resultantes sean lo más puros posible.

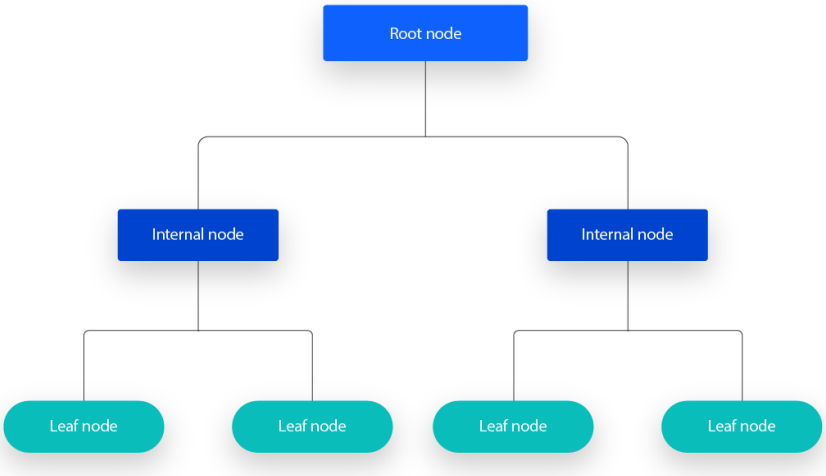


Figure 1: La idea básica de un árbol de decisión.

Las características descriptivas que mejor reproducen las características objetivo normalmente se consideran las más informativas. El proceso de encontrar la característica más informativa se lleva a cabo hasta que se cumple un criterio de detención, momento en el cual llegamos a los llamados **nodos hoja**.

Un árbol de decisión generalmente se divide en un **nodo raíz**, los nodos interiores y los nodos hoja finales, o simplemente **hojas**. Estas entidades están conectadas mediante las llamadas **ramas**.

Los nodos hoja contienen las predicciones que realizaremos para nuevas instancias de consulta presentadas a nuestro modelo entrenado. Esto es posible porque

el modelo ha aprendido la estructura subyacente de los datos de entrenamiento y, por lo tanto, puede, dado ciertos supuestos, **hacer predicciones** sobre el valor de la característica objetivo (clase) de instancias de consulta no vistas.

El enfoque general de los árboles de decisión es un enfoque de arriba hacia abajo. Una instancia se clasifica comenzando en el nodo raíz del árbol, probando el atributo especificado por este nodo, y luego bajando por la rama del árbol que corresponde al valor del atributo en el ejemplo dado. Este proceso se repite para el subárbol enraizado en el nuevo nodo.

3 Construyendo el árbol de decisión

En este ejercicio utilizarás `DecisionTreeClassifier` de Scikit-learn para construir un árbol de decisión para el conjunto de datos del vino (`https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html`).

Primero, crearemos un objeto `DecisionTreeClassifier` con la configuración predeterminada (es decir, sin pasar ningún parámetro al constructor), y lo entrenamos usando las variables del conjunto de datos del vino que hemos definido.

```
from sklearn.tree import DecisionTreeClassifier
X = dataset["data"]
y = dataset["target"]
DT = DecisionTreeClassifier()
DT.fit(X, y)
```

Felicidades, has creado un árbol! Ahora vamos a visualizarlo. Scikit-learn ofrece una función `plot_tree()`, que muestra el árbol.

```
from sklearn.tree import plot_tree
plt.figure(figsize=(50, 20))
plot_tree(DT, feature_names=dataset["feature_names"],
class_names=dataset["target_names"], filled=True)
plt.show()
```

Una alternativa es usar `export_graphviz()`. Esta función exporta el árbol como un archivo DOT, un lenguaje para describir gráficos. Desde el código DOT puedes generar la representación visual resultante con el siguiente fragmento de código:

```
from sklearn.tree import export_graphviz
import pydot
from IPython.display import Image
dot_code = export_graphviz(DT, feature_names=dataset["feature_names"],
class_names=dataset["target_names"])
graph = pydot.graph_from_dot_data(dot_code)
Image(graph[0].create_png())
```

Ahora, analizaremos el árbol: fíjate en cómo **cada división genera dos ramas**: como se puede ver desde la división raíz, la convención utilizada es que la rama más a la izquierda es la que se selecciona cuando la comparación da un valor "**verdadero**", y la rama más a la derecha cuando la comparación es "**falsa**".

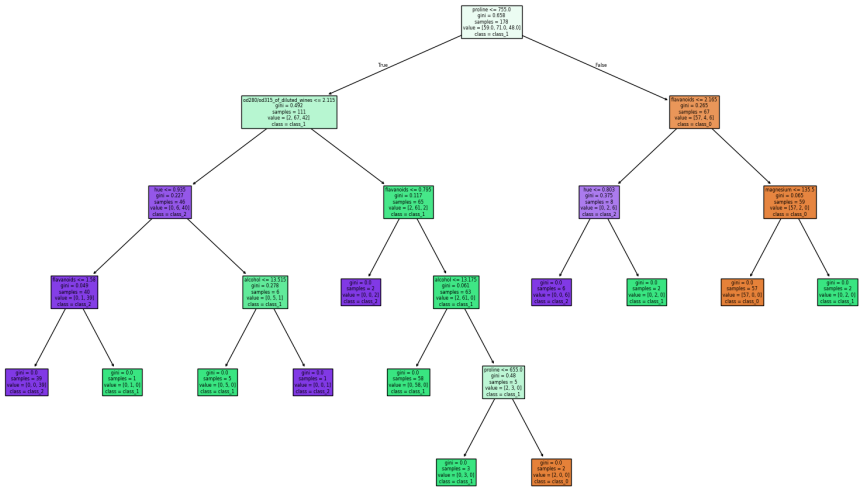


Figure 2: Tu árbol de decisión.

Cada uno de los nodos del árbol contiene cierta información, y puedes analizar el árbol en más detalle.

- El número de muestras que se encuentran en cada nodo. En el nodo raíz, comenzamos con 178 muestras, y en cada nodo se especifica cuántos puntos del conjunto de datos alcanzan ese nodo. `class = class_1`: este valor indica que el modelo ha asignado esta clase, en este caso, `class_1`, como la clase mayoritaria para el nodo raíz.

- El atributo utilizado para la división y el valor umbral usado en este primer nodo es `proline <= 755.0`.

- Se utiliza el **índice Gini** como un indicador de qué tan puro es cada nodo:

$$Gini = 1 - \sum_{i=1}^{k} p_i^2$$

  donde k es el número de clases, p_i es la proporción de elementos que pertenecen a la clase i. En problemas binarios, el índice Gini varía entre

0 y 0.5. Para el caso de tres clases:

$$Gini = 1 - p_1^2 - p_2^2 - p_3^2$$

y el índice Gini máximo es 0.667. Para un problema de N clases, el índice Gini varía de 0 a $(1 - \frac{1}{N})$.

- Calculemos el índice Gini en el nodo raíz:

$$Gini = 1 - \left(\frac{59}{178}\right)^2 - \left(\frac{71}{178}\right)^2 - \left(\frac{48}{178}\right)^2$$

- Nodo raíz: `proline <= 755.0`. La condición es: si el valor de proline es menor o igual a 755, se sigue la rama izquierda; si es mayor a 755, se sigue la rama derecha. Vamos a observar la rama **derecha**, donde `proline > 755.0`.

- Siguiente nodo: `flavanoids <= 2.165`. La condición es: si el valor es menor o igual a 2.165, se sigue la rama izquierda de este nodo. Si es mayor que 2.165, se sigue la rama **derecha**.

- Siguiente nodo: `magnesium <= 135.5`. La condición es: si el valor es menor o igual a 135.5, se sigue la rama izquierda. Si es mayor que 135.5, se sigue la rama **derecha**.

- Nodo hoja: `class = class_1`. Finalmente, llegamos a un nodo hoja con 2 muestras, donde todas las muestras pertenecen a la clase 1.

- En los nodos intermedios, el Gini es mayor que 0 porque hay mezcla de clases. En los nodos finales, el Gini es 0 porque solo hay una clase, lo que indica pureza total. Un árbol bien entrenado busca crear nodos lo más puros posible en cada paso para lograr una mejor clasificación.

- Nota que algunos de los nodos están "tomando decisiones" basadas en pequeñas porciones del conjunto de datos de entrenamiento. Por ejemplo, la división `flavonoides <= 1.58` separa 39 elementos de la clase "2" de un solo elemento de la clase "1".

- Observa que varias características se usan en múltiples ocasiones. Por ejemplo, la característica `flavonoides` se utiliza 3 veces en diferentes caminos de raíz a hoja, lo cual está bien, ya que el árbol no "descarta" las características que ya ha utilizado.

- La característica `alcohol` se usa dos veces en el mismo camino de raíz a hoja (`alcohol <= 13.175` y `alcohol <= 13.515`). En estos casos, los diferentes umbrales están definidos de manera compatible: dado que la división `alcohol <= 13.515` está a la derecha del nodo anterior, significa que el valor de `alcohol` ya es mayor que 13.175. Esto implica que los elementos en la hoja izquierda tendrán un valor de `alcohol` que está entre 13.175 y 13.515.

4 Predicciones

Ahora que hemos analizado visualmente el arbol de decision, usa la función `predict()` de `DecisionTreeClassifier` para obtener las predicciones del clasificador. Luego, usa `accuracy_score()` para calcular la precisión entre la lista de etiquetas correctas y la lista de predicciones hechas por el clasificador.

```
predictions = DT.predict(X)
acc = accuracy_score(y, predictions)
print("Precisión:", acc)
```

El resultado es 1: es decir, 100% de precisión. La razón por la que estamos obteniendo una precisión tan alta es que estamos **probando nuestro modelo con los mismos datos** que usamos para el entrenamiento. No es particularmente útil calcular la precisión con los mismos datos que usamos para el entrenamiento, ya que eso no nos dirá **nada sobre qué tan bien nuestro clasificador generaliza a nuevos datos**.

5 División del conjunto en train y test

Lo que vamos a hacer ahora es dividir el conjunto de datos en un conjunto de entrenamiento y uno de prueba usando `train_test_split()`, y evaluar cómo la forma en que se dividen los datos de un conjunto de clasificación afecta la distribución de clases y, en consecuencia, el rendimiento del modelo.

1. Divide los datos usando:

   ```
   from sklearn.model_selection import train_test_split
   ```

   Utiliza un 80% de los datos para entrenamiento y un 20% para prueba.

2. Las opciones del método admiten:

   ```
   train_test_split(*arrays, test_size=None,
   train_size=None, random_state=None, shuffle=True, stratify=None)
   ```

   y vamos a usar la opción: shuffle=**False**

3. Crea los sub-conjuntos `X_train`, `y_train`, `X_test` y `y_test`.

4. Cuenta la cantidad de muestras de cada clase en los conjuntos de entrenamiento y prueba, y calcula los porcentajes de cada clase en ambos conjuntos.

5. Entrena el modelo y calcula la precisión. Ya no es 1?

```
X_train1, X_test1, y_train1, y_test1 = train_test_split(X, y, test_size=0.2, shuffle=False,
clf1 = DecisionTreeClassifier()
clf1.fit(X_train1, y_train1)
acc1 = accuracy_score(y_test1, clf1.predict(X_test1))
```

6 Evaluación de métricas

Para calcular otras métricas (por ejemplo, precisión, recall y F1-score), se usan las funciones respectivas (`precision_score`, `recall_score`, `f1_score`).

```
from sklearn.metrics import precision_score, recall_score, f1_score, classification_report
```

Ten en cuenta que, dado que estas tres métricas están basadas en una sola clase, puedes calcular el valor para una clase específica, agregar los resultados en un solo valor o recibir los resultados para todas las clases. Consulta el parámetro `average` en la documentación para obtener más información sobre esto.

También puedes usar la función `classification_report`, que devuelve varias métricas (incluyendo las mencionadas anteriormente) para cada una de las clases del problema.

7 Tarea

Volver a entrenar el conjunto como se hizo en la Sección 5 pero ahora:

1. Prueba tres maneras diferentes:
   1) sin mezclar: `shuffle=False`
   2) con mezclar: `shuffle=True`
   3) con estratificación: `stratify=y`

2. Crea los sub-conjuntos `X_train`, `y_train`, `X_test` y `y_test`, respectivamente. Cuenta la cantidad de muestras de cada clase en los conjuntos de entrenamiento y prueba. Calcula los porcentajes de cada clase en ambos conjuntos para las tres opciones de división.

3. Entrena un modelo para cada una de las tres opciones y calcula la precisión. Compara y analiza estos valores con la precisión obtenida previamente.

4. Estudia también las métricas: *precision*, *recall* y *F1-score*.

5. ¿Cuál es el valor más razonable? ¿Por qué? ¿En qué casos es crítico utilizar `stratify=y` y ¿cuándo no es tan importante?